Can Coding Agents Produce Merge-Ready Feature Pull Requests? An Industrial Audit of 62 Real Tasks

Zihan Wu*
Department of Electrical Engineering, City University of Hong Kong
Hong Kong, China

Yun Peng*,†
Department of Computer Science and Engineering, The Chinese University of Hong Kong
Hong Kong, China
ypeng@cse.cuhk.edu.hk

Chun Yong Chong
School of Information Technology, Monash University Malaysia
Subang Jaya, Malaysia

Xin Zhou
Singapore Management University
Singapore, Singapore

Zeyang Zhuang
The Chinese University of Hong Kong
Hong Kong, China

Rui Shu
Independent Researcher
Singapore, Singapore

Kui Liu
Huawei Technologies Co., Ltd.
Shenzhen, China

Abstract

Industrial teams now face a deployment question, not a benchmark question: *can current coding agents produce feature pull requests*—multi-file, API-changing, test-bearing changes—*that engineers would actually merge?* We answer this for Huawei's rollout by auditing four leading commercial agents on 62 real feature tasks: 12 from Huawei's CANN/`torch_npu`, MindSpeed, and Kubernetes work, plus 50 from our open-source extension CapBench. Across $4 \times 62 \times 2 = 496$ runs (long and short prompts) judged by a four-LLM panel calibrated against three blinded human reviewers (92.7% PASS-vs-non-PASS agreement, panel runs strictly stricter), only 1.6% of runs reach a merge-ready verdict, and the four Kubernetes tasks return 0 of 32 PASS for every agent. We harden the audit against ground-truth leakage after observing one agent cherry-picking the target diff from upstream history. The 1.6% headline is not the contribution: the contribution is the deployment policy that follows—route only bounded, single-module feature work to agents today, gate every draft through a multi-judge harness with explicit scope and coverage checks, and keep cross-cutting changes (Kubernetes-class controllers, protocol-level refactors) under human authorship until those gates close on a measurable failure mode.

ACM Reference Format:
Zihan Wu*, Yun Peng*,†, Chun Yong Chong, Xin Zhou, Zeyang Zhuang, Rui Shu, and Kui Liu. 2026. Can Coding Agents Produce Merge-Ready Feature Pull Requests? An Industrial Audit of 62 Real Tasks. In . ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

*Equal contribution. †Corresponding author: ypeng@cse.cuhk.edu.hk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Software engineering organizations are under growing pressure to turn coding agents from research demonstrations into deployable engineering infrastructure. The decision they face is not whether an agent can suggest code in an IDE, but whether it can produce a reviewable pull request for substantive feature work in a production codebase. At Huawei, this question is operational: multiple stack-level teams—Kubernetes, the CANN compiler stack (CANN), the MindSpeed training framework, and `torch_npu`—are deciding in 2026 whether to route portions of their feature backlog through coding agents. The deployment decision turns on a concrete question: *when an agent is given a feature specification and the corresponding repository, can it produce a candidate PR that a human reviewer would accept with at most light revision?*

Why existing benchmarks do not answer this question. Coding-agent benchmarks have made fast progress on bug-fix and short-task settings. Industry-watched leaderboards report that the strongest agent configurations now resolve over 70% of issues on SWE-bench Verified [6], and many leaderboards exist for short, well-localized tasks. But these tasks are predominantly bug repair: they target a small set of files identified in advance, often supply the failing test, and reward minimal local edits. Feature implementation differs in kind, not degree. A new feature requires the agent to identify the right architectural surface, modify several files in coordinated ways, add or update tests, respect repository-specific conventions, and produce a change set that integrates with parallel ongoing work. Industry surveys (e.g., [4]) confirm that feature work—rather than localized bug fixes—is the bottleneck developers most want help with. The gap between bug-repair benchmark scores and merge-ready feature pull requests is therefore not just a question of model size; it is a question about whether today's agents can deliver value on the part of the workflow that matters most to engineering teams.

An industrial audit. We answer the deployment question directly by auditing four leading commercial coding agents on a curated 62-task feature-implementation benchmark. The benchmark combines 12 *Huawei industrial tasks* drawn from the codebases that the rollout would touch (Kubernetes, the CANN/`torch_npu` accelerator stack, and MindSpeed) with 50 *open-source feature tasks* from a curated extension we call CapBench, drawn from 15 production-grade open source projects (Kafka, CPython, Airflow, TensorFlow, Apache Spark, Postgres, ClickHouse, etc.). Each task is paired with a *long* prompt (a multi-paragraph design proposal that mirrors a real feature spec) and a *short* prompt (a one-sentence ticket-style description), giving us $62 \times 2 = 124$ task instances per agent. We evaluate four agents—Claude Code (Opus 4.6), OpenAI Codex (GPT-5.4), Cursor Composer-2, and OpenCode/GLM-5.1—producing $124 \times 4 = 496$ end-to-end agent runs. Each run is independently scored on three production-relevant axes (functional correctness, completeness/coverage, and behavioral equivalence to the reference implementation) by an automated four-evaluator panel; the verdict is decided by 3-of-4 majority. We treat the four agents not as four versions of the same model but as four *harnesses*: each ships its own planning loop, tool integration, code-search retrieval, and verification logic, and our results suggest these harness choices account for a large share of the variation we observe across vendors (Section 5).

This setup gives us a clean answer to the question Huawei's deployment teams asked: *can today's coding agents take over feature delivery for our internal codebases?*

Why the verdict can be trusted. Two design choices preempt the obvious objections to LLM-judged benchmarks. *(i) Human-rater calibration.* We blind-rated the 96 Huawei-internal runs with three senior reviewers and confirmed 92.7% PASS/non-PASS agreement with the four-LLM judge panel; the panel is the stricter of the two, never crowning a human-rejected run as PASS. *(ii) Anti-leakage sanitization.* An early Codex run on a Kubernetes task surfaced its ground-truth diff via `git log`; we treated this as a first-class threat, rebased every benchmark repo to a parent commit with target commits unreachable, blocked network access to upstream remotes, and re-ran the affected configurations. The numbers we report are after this re-run. Both procedures are detailed in §2.

Headline findings. Three findings shape the deployment recommendation:

- **Aggregate verdict: 8 of 496 runs pass strict majority review (1.6%).** Treating the four evaluators as a panel and requiring 3-of-4 PASS, only 8 runs reach production-equivalent quality. The remaining 488 runs are split across PARTIAL (194 runs, 39%) and FAIL (294 runs, 59%).
- **Harness choices appear to influence outcomes as much as model choice.** The two stronger harnesses (Claude Code and Codex CLI) reach 2.4% PASS each, while Cursor Agent and OpenCode CLI reach 0.8% PASS each—a 3× spread on identical tasks and prompts. Because every agent couples a specific harness to a specific frontier model, we cannot isolate harness from model; the comparison is product-level. We treat this as evidence that planning loop, retrieval, file resolution, and edit verification carry weight comparable to model choice in this regime, not as a clean harness-vs-model ablation.
- **Long-prompt advantage is asymmetric.** On long prompts (with full design context), the strong harnesses lift PASS rate from 0% to 4.8%; the weaker harnesses improve only to 1.6%. Better specification helps strong harnesses ship; for weak harnesses, more context improves scores but rarely converts outputs into production-equivalent PRs.
- **Kubernetes-class tasks are still out of reach.** All 4 harnesses produce 0/8 PASS on the four K-tasks (Kubernetes scheduler / kubelet feature work). Cross-cutting feature work in mature Go infrastructure remains the hardest deployment frontier we measured.
- **Failures are heterogeneous, but many are harness-addressable.** The 488 non-PASS runs split into semantic/correctness failures (41.4%), coverage gaps (26.2%), behavioral/design mismatches (10.0%), and borderline partials (22.3%). The important deployment signal is not that every failure is mere incompleteness, but that a large recoverable slice involves stopping early, omitting tests or generated artifacts, or failing to wire cross-module changes.

Contributions. This paper makes three industrial-deployment contributions. *(1) Industrial audit protocol.* An audit protocol for feature-PR readiness that combines a 62-task benchmark (12 Huawei production tasks plus 50 OSS CapBench tasks), a four-LLM judge panel with strict majority verdicts, a repository sanitisation procedure that survived an attempted ground-truth exfiltration, and a 96-run blind human calibration ($\kappa = 0.43$, 92.7% agreement on PASS/non-PASS). *(ii) Empirical capability boundary.* Across 496 runs we find that today's commercial coding agents reach merge-ready quality on at most 2.4% of tasks; the entire Kubernetes track is 0/32; long-form specifications double the strongest harnesses' PASS rate but do not move the weaker ones; and the failure mass is dominated by semantic errors (41.4%) rather than incomplete coverage—a result that contradicts the common assumption that better retrieval would close the deployment gap. *(iii) Deployment policy.* The audit drives a concrete rollout decision used by Huawei's deployment review: ship agents only on bounded operator-tier work behind multi-judge harness gates (scope/test/judge), keep cross-module platform work in human hands, and treat the long-form specification template plus the calibrated panel as required infrastructure rather than research artifacts.

2 Study Design

Figure 1 summarizes our experiment pipeline. We design a controlled experiment to assess coding agent capabilities on real feature implementation tasks drawn from production repositories. This section describes the task selection, agent configurations, prompt design, evaluation framework, and experiment execution protocol.

2.1 Task Selection

Our benchmark has 62 tasks in total: 12 from the Huawei/platform ecosystem (Kubernetes, the CANN/`torch_npu` accelerator stack, and MindSpeed) and 50 from *CapBench*, an open-source extension assembled from 15 widely used community projects (Kafka, CPython, Airflow, Terraform, Spring, etc.). Each task corresponds

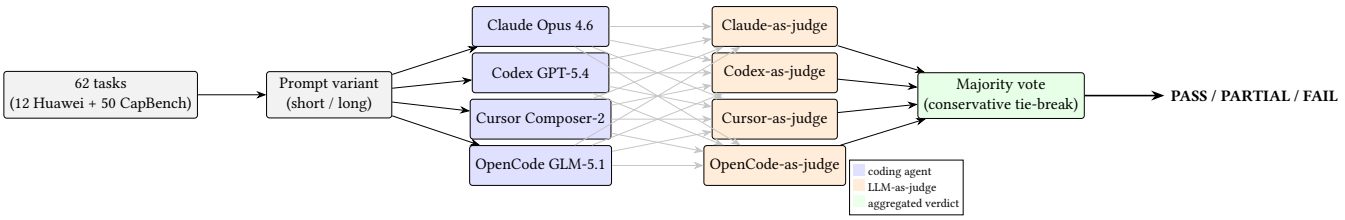

Figure 1: Evaluation pipeline. 62 tasks × 2 prompt variants × 4 agents yield 496 runs; each run is scored by four LLM judges (including the same model families as the agents themselves), then combined by majority vote with a conservative tie-break favouring PARTIAL over PASS. The pipeline produces 1,984 evaluator judgments in total.

to a single merged pull request or commit implementing one coherent feature, bug fix, or refactor. Table 1 summarises the Huawei/platform subset; the full CapBench task inventory is included in the replication package.

Selection criteria. Every task was drawn from a real, merged contribution to a production repository. C-family accelerator tasks (C1–C5) and MindSpeed tasks (M1–M3) come from Huawei's Ascend NPU software stack—operator kernels, `torch_npu` integration, training framework features, and precision fixes that represent typical industrial workloads. Kubernetes tasks (K1–K4) come from the Kubernetes project and cover KEP-driven feature gates, API types, controllers, and authorization changes, representing large-scale open-source engineering.

Complexity classification. We classify tasks into three tiers based on the number of ground-truth files, cross-component coordination requirements, generated-code or API surface, and domain-specific knowledge needed. The thresholds are not pure file-count bins: a small Kubernetes API change can be High because it requires generated types, validation, tests, and release-cycle knowledge, while a bounded Python change can remain Medium even with a larger diff.

- *Low* (C1, M1): localised edits with limited architectural coupling.
- *Medium* (C2, C3, M2, M3): bounded multi-file changes or domain-specific implementation work that remains inside a narrow subsystem.
- *High* (C4, C5, K1–K4): cross-component changes that require architectural understanding, generated/API surface updates, multi-module orchestration, or extensive testing.

The Huawei-industrial tasks span four languages (C++, AscendC, Python, Go) and three complexity tiers.

CapBench extension (50 tasks). To broaden beyond the Huawei ecosystem, we added 50 tasks from CapBench, our open-source complementary suite curated from 15 widely-used projects: Kubernetes, Kafka, Spring Boot, CPython, Django, Airflow, TensorFlow, Apache Flink, LangChain, Kubernetes Operators, Redis, and others. Tasks are drawn from merged PRs that implement a real feature or fix, span 1–127 handwritten files, and cover six languages (Go, Python, Java, Rust, TypeScript, C++). CapBench tasks use the same metadata rubric: Low tasks are local edits, Medium tasks are bounded multi-file changes, and High tasks require broader cross-component coordination. Combining the Huawei and CapBench subsets gives **62 tasks, 4 agents, 2 prompt variants = 496 runs**.

2.2 Agent Configurations

We evaluate four primary agent configurations, summarised in Table 2. Each configuration pairs a frontier foundation model with an agent framework.

Product-level comparisons. Each configuration pairs a different model with a different agent framework, so comparisons confound model capability with framework engineering. These comparisons are best understood as *product-level* assessments—comparing the capabilities of commercially available agent tools—rather than controlled model ablations. We discuss this confound further in §7.

2.3 Prompt Design

Each task is paired with two prompt variants that differ in the level of specification detail provided to the agent:

- **Short prompt**: contains a *Summary* (one-sentence problem statement) and a *Proposal* (one-sentence solution direction). Typically 2–4 sentences.
- **Long prompt**: extends the short prompt with *Motivation* (why the change is needed) and *Design Details* (numbered list of implementation considerations). Typically 15–30 sentences.

Design principle. Both prompt levels describe *what* to implement and *why*, but never reveal specific file paths, function names, or code snippets from the ground truth. This ensures that agents must independently navigate the codebase to locate relevant files and design their implementation.

Example (C1, buffer alias fix). The short prompt reads: "*ScaledMaskedSoftmaxGradV2 has a tensor-lifetime bug: a buffer is used after its queue resource is freed. Replace the reference with an equivalent surviving buffer.*" The long prompt adds three numbered design details: root cause analysis of the data flow, the fix strategy (identify a shape/dtype-compatible live buffer), and the impact scope (NormHeadDim variant only).

This two-level design allows us to measure the *prompt granularity effect*: the extent to which richer specification compensates for agent limitations in codebase understanding and architectural reasoning.

2.4 Evaluation Framework

Ground truth. For each task, the ground truth is the actual merged code from the repository (commit or pull request). This provides a

Table 1: Huawei/platform task registry (12 of the 62 tasks used in this study). GT = ground truth (merged code). Complexity is the three-tier metadata used in the evaluation CSV and reflects file count, cross-component coordination, generated-code surface, and domain-specific knowledge. The remaining 50 tasks come from the CapBench open-source suite.

ID	Repository	Complexity	Lang	GT Files	GT Lines
C1	cann-ops-adv	Low	C++	1	3
M1	MindSpeed	Low	Python	3	19
C2	cann-ops	Medium	C++	4	9
C3	torch_npu	Medium	Python	9	637
M2	MindSpeed	Medium	Python	6	150
M3	MindSpeed	Medium	Python	10	948
C4	cann-ops	High	AscendC	24	1,273
C5	cann-ops	High	AscendC	27	3,372
K1	kubernetes	High	Go	13	1,010
K2	kubernetes	High	Go	35	3,759
K3	kubernetes	High	Go	49	11,000
K4	kubernetes	High	Go	98	6,507

Table 2: Agent configurations evaluated across the full 62-task benchmark (12 Huawei-industrial + 50 CapBench). Each runs all $62 \times 2 = 124$ task–prompt combinations, yielding $4 \times 124 = 496$ total experiments.

Config	Model	Framework	N
Claude	Claude Opus 4.6	Claude Code	124
Codex	GPT-5.4	Codex CLI	124
Cursor	Composer-2	Cursor Agent	124
OpenCode	GLM-5.1	OpenCode	124

concrete, human-reviewed reference implementation against which we compare agent output.

Scoring dimensions. Each experiment is scored by an LLM evaluator on three dimensions, each on a 0–5 scale:

- **Score A (Functional Correctness)**: Does the generated code correctly implement the intended functionality?
- **Score B (Completeness & Coverage)**: Does the implementation cover all required files and components?
- **Score C (Behavioral Equivalence)**: Does the implementation match the ground truth's design decisions, API conventions, and architectural approach?

Verdict rules. We assign a three-level verdict based on the dimension scores:

- PASS: $A \geq 4$ **and** $B \geq 4$ **and** $C \geq 3$.
- FAIL: $A < 2$ **or** $B < 2$ **or** $C < 2$ (any dimension critically deficient).
- PARTIAL: all other cases.

File coverage metric. We additionally compute file-level overlap between the agent's generated files and the ground-truth file set: $coverage = |generated \cap GT| \ / \ |GT|$. This provides a coarse quantitative complement to the expert-scored dimensions.

Evaluation scope. Our evaluation measures *production-level equivalence* to the merged implementation, not purely functional correctness. An agent solution that correctly addresses the requirement but uses a materially different file organization, naming convention, or architecture than the accepted PR will score lower on Behavioral Equivalence (C) and file coverage. This design choice reflects our industrial interest: we assess whether agent output could replace a human-authored PR draft, not merely whether it contains a plausible local fix. As a consequence, our FAIL verdicts should be read as "not production-equivalent" rather than "functionally impossible." Our root-cause analysis (§4) confirms this: many non-PASS cases stem from engineering completeness gaps rather than total semantic misunderstanding.

Multi-evaluator cross-validation. To mitigate evaluator bias, every experiment is independently scored by four LLM evaluators from different model families: Claude Opus 4.6, Codex GPT-5.4, Cursor Composer-2, and GLM-5.1. We report the *majority verdict* (strict majority where possible, with a conservative tie-break that orders labels FAIL<PARTIAL<PASS and selects the lower label when evaluators are split) and scores averaged across the four. Unanimity on the verdict label holds for 33.7% of experiments; pairwise verdict agreement ranges from 42.5% (Codex vs GLM, the two strictest evaluators) to 74.4% (Claude vs Cursor). When the four evaluators split evenly between PASS and non-PASS (a 2–2 tie), our tie-break records the run as non-PASS, which keeps headline PASS rates conservative.

Human pilot calibration. To check that the LLM panel is not idiosyncratically biased relative to domain-expert review, we conducted a blind human-rater pilot on all 96 Huawei runs (12 tasks $\times$ 4 agents $\times$ 2 prompts). Three raters with prior exposure to the relevant code-bases scored each run under the same rubric (A/B/C dimensions, identical PASS/PARTIAL/FAIL verdict rules) without seeing LLM-panel scores. On these 96 cells the human verdicts came back as **9 PASS / 55 PARTIAL / 32 FAIL**, vs the LLM-panel **4 PASS / 57 PARTIAL / 35 FAIL** under our 3-of-4 majority rule; raw three-way verdict agreement is 66.7% (Cohen's $\kappa = 0.38$), and PASS-vs-non-PASS agreement is 92.7% ($\kappa = 0.43$). The LLM panel is the stricter of the two: 6 of the 9 human-PASS cells were demoted by the panel (4 to PARTIAL, 2 to FAIL), and the panel never overturned

a human-FAIL into a PASS. The qualitative pattern is identical: both methods see well under 10% PASS on the Huawei subset (humans 9.4%, panel 4.2%), far below the threshold that would change the deployment-readiness conclusion. We treat this as evidence that the 1.6% headline PASS rate over the full 496-run benchmark is a conservative *lower* bound on what a careful human reviewer would mark merge-ready, not a strict-evaluator artifact: humans were more lenient than the panel on a handful of borderline runs, never harsher. Per-rater rationales are released in the artifact bundle.

2.5 Experiment Execution

Each (task, agent, prompt) cell runs once in a fresh sandbox built per task. The repository is rebased to the parent commit of the merged PR, with upstream remotes removed and `gh`/`git fetch` shimmed off, so the agent cannot recover ground truth from history. We discovered the importance of the second precaution after an early Codex run on K4 reached the merged patch via `git log`; we now treat anti-leakage as a benchmark requirement rather than an afterthought.

3 Results

We organise the audit results around the deployment question: of 496 runs (62 tasks × 4 agents × 2 prompts), how many produce a candidate PR a senior engineer would actually merge, and on which task shapes? The headline answer is *very few*: 8 PASS in 496 (1.6%), the best harness reaches 2.4%, and no agent clears any Kubernetes-family task.

Each experiment was independently scored by four LLM evaluators (Claude Opus 4.6, Codex GPT-5.4, Cursor Composer-2, GLM-5.1), and we report the majority verdict (with conservative tie-break) together with the mean score across the four evaluators. Pairwise evaluator agreement ranges from 42.5% (Codex vs. GLM) to 74.4% (Claude vs. Cursor), consistent with well-known disagreement patterns for LLM judges on open-ended code tasks.

3.1 Agent-Level Performance

Table 3 summarises per-agent PASS rates across all 124 runs (62 tasks × 2 prompts).

Figure 2 further breaks down PASS by task family, confirming that the Kubernetes family yields zero PASS across all agents.

Finding: Headline result. *Across 496 runs (62 tasks × 4 agents × 2 prompts), only 8 clear the full rubric (1.6% PASS). The strongest configurations are Claude Opus 4.6 and Codex GPT-5.4 at 2.4%; Cursor Composer-2 and OpenCode GLM-5.1 each pass only 0.8%. No agent passes any Kubernetes-family task, and no agent exceeds 4.8% PASS even on its best prompt variant.*

Finding 1 (Task shape dominates over agent product). The spread between the strongest and weakest agent in our test (2.4% vs. 0.8%) is small compared to the spread induced by task family (4.8% on long-prompt CapBench tasks vs. 0% on every Kubernetes task). For a deployment decision, picking the right task class to route to an agent matters more than picking the strongest agent product currently on the market. We return to this observation in §5.

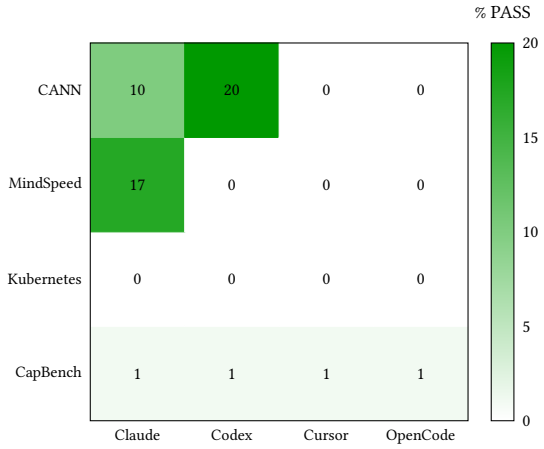

Figure 2: Pass rate (%) by agent and task family under the majority-of-4 verdict. Rows top-to-bottom: *CANN, MindSpeed, Kubernetes, CapBench*. Kubernetes (K1–K4) is a *uniform zero-pass* cliff across all agents.

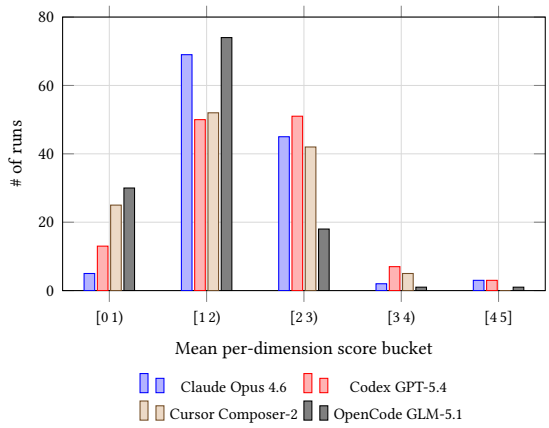

Figure 3: Per-run score distribution across four agents (n=124 runs each). Each bucket uses the mean per-dimension score, i.e., $(A+B+C)/3$ averaged across the four judges on a 0–5 scale; Table 3 reports the corresponding summed 0–15 composite mean. Most runs fall in the 2–3 bucket across all agents, reflecting partial-pass behavior; full-rubric PASS requires scores well above 4.

3.2 Task-Family Effect

Our benchmark spans four task families: CANN/`torch_npu` accelerator stack tasks (C1–C5), MindSpeed distributed training (M1–M3), Kubernetes-family (K1–K4), and CapBench OSS (T01–T50). The PASS distribution differs dramatically across families:

Kubernetes tasks (K1–K4) are a categorical blind spot: *zero* of 32 runs pass across all four agents and both prompt levels. C-family accelerator-stack tasks achieve the highest PASS rate (7.5%), reflecting that Huawei-specific operator and `torch_npu` integration changes are often more localised than Kubernetes platform work.

Table 3: Per-agent PASS/PARTIAL/FAIL distribution across all 496 runs. Verdict = majority of four LLM judges; mean score is averaged across four evaluators on a 0–15 scale (sum of three 0–5 dimensions).

Agent	N	PASS	PART.	FAIL	PASS%	Mean
Claude (Opus 4.6)	124	3	57	64	2.4%	5.68
Codex (GPT-5.4)	124	3	62	59	2.4%	**5.82**
Cursor (Composer-2)	124	1	49	74	0.8%	4.88
OpenCode (GLM-5.1)	124	1	26	97	0.8%	4.16
All agents	496	8	194	294	1.6%	5.14

Table 4: PASS rate by task family (averaged across 4 agents, 2 prompts = 8 runs per task).

Family	Repos	Tasks	Runs	PASS%
C (Huawei CANN/`torch_npu`)	cann-ops, cann-ops-adv, torch_npu	5	40	7.5%
M (MindSpeed, Python)	MindSpeed	3	24	4.2%
K (Kubernetes, Go)	kubernetes	4	32	0.0%
T (CapBench, mixed)	15 OSS projects	50	400	1.0%

CapBench OSS tasks average 1.0% PASS, consistent with the overall 1.6% but revealing that Kubernetes-class difficulty scales with project size.

Finding 2 (Kubernetes Cliff). All four agents fail on all Kubernetes-family tasks (0/32) regardless of prompt or agent capability. Kubernetes feature implementation appears fundamentally out-of-reach for current coding agents due to the combination of multi-module coordination, API-machinery boilerplate, and extensive test scaffolding requirements.

3.3 Per-Task Pass Distribution

Of the 62 tasks, only 4 have any PASS across the 4 agents: C1, C4, M1, and T10. The rerun of disputed Cursor cases removes the previous C1-short, C4-short, and T34-long Cursor PASS outcomes. Table 5 summarises the per-task PASS distribution.

Every other task has 0 PASS across all 8 attempts (4 agents × 2 prompts). Notably, of the 50 CapBench OSS tasks, only T10 elicits any PASS at all; all Huawei-ecosystem tasks except C1, C4, and M1 score 0 PASS.

Task concentration. The 8 PASS outcomes are concentrated in four tasks rather than spread across the benchmark. Codex and Claude tie at 3 of 124 (2.4%), while Cursor and OpenCode pass only T10-long (1 of 124, 0.8%). Codex leads by mean composite score (5.82/15, vs. 5.68 for Claude and 4.88 for Cursor), indicating that it more often produces useful PARTIAL outputs even when it does not reach full production equivalence.

3.4 Complexity-driven failures

Complexity tiers for the 12 industrial tasks are assigned by the curation metadata used in the evaluation CSV: Low (C1, M1), Medium (C2, C3, M2, M3), and High (C4, C5, K1–K4). CapBench tasks span broader ranges but lean toward High. Results:

On the broader CapBench extension (50 tasks × 4 agents × 2 prompts = 400 runs), only 4 runs reach PASS: Claude, Codex, Cursor, and OpenCode on T10-long. The effect is even starker: the 50 CapBench tasks collectively yield 1.0% PASS rate.

Finding 4 (updated): *Task complexity creates a near-absolute capability cliff.* Pass rate drops from 18.8% on Huawei Low tasks to 0% on Medium tasks and 2.1% on High tasks, where the only PASS is a single CANN run rather than Kubernetes-scale platform work. The CapBench subset pushes the aggregate to 1% PASS even for the best agents.

3.5 Prompt Effect (Long vs Short)

Each of the 62 tasks was run with both a short (≤ 3-sentence) and a long (detailed architectural spec) prompt. Table 7 aggregates pass rates and mean scores by agent and prompt type.

For Claude and Codex, detailed prompts yield a +4.8-point PASS-rate improvement and a ∼+1.2-point mean-score boost. Cursor and OpenCode each gain only one long-prompt PASS, remaining near-zero in absolute terms.

Finding 3: Longer prompts help all four configurations on mean score, but only Claude and Codex convert that extra specification into multiple PASS outcomes. This suggests that richer problem specifications amplify existing model capability rather than making weaker configurations production-equivalent.

3.6 Evaluator Agreement

Pairwise verdict agreement among the four evaluators spans from 42.5% (Codex-GLM, the strictest pair) to 74.4% (Claude-Cursor, the most aligned).

Codex and GLM agree the least (42.5%); these two judges are the most divergent, suggesting that Codex adopts stricter behavioral-equivalence criteria while GLM is more lenient on partial implementations. Claude and Cursor agree most often (74.4%). Full 4-way consensus occurs on 33.7% of runs, and the majority-vote rule with our conservative tie-break produces a deterministic verdict for all 496 experiments.

Finding 5. Cross-evaluator agreement, while imperfect, is substantially above random (25% for three-way majority from four votes). The 42.5% Codex–GLM agreement implies that single-evaluator benchmarks are noisy on borderline agent outputs;

Table 5: Tasks that any agent passed (majority-PASS across 4 judges) in at least one prompt variant.

Task	Agents passing (and prompt)	Total PASS (of 8 runs)
C1 (CANN)	Claude (long), Codex (long)	2
C4 (CANN)	Codex (long)	1
M1 (MindSpeed, Low)	Claude (long)	1
T10 (CapBench)	Claude, Codex, Cursor, OpenCode (all long)	4
Total	8 / 496	1.6%

Table 6: PASS rate by complexity tier, for the 12 Huawei industrial tasks.

	Codex	Cursor	Claude	OpenCode	Avg
Low (2 tasks)	1/4 (25.0%)	0/4 (0%)	2/4 (50.0%)	0/4 (0%)	3/16 (18.8%)
Medium (4 tasks)	0/8 (0%)	0/8 (0%)	0/8 (0%)	0/8 (0%)	0/32 (0%)
High (6 tasks)	1/12 (8.3%)	0/12 (0%)	0/12 (0%)	0/12 (0%)	1/48 (2.1%)

Table 7: Prompt effect. Δ = long − short, percentage points.

Agent	Short PASS	Long PASS	ΔPASS	Short mean	Long mean	Δmean
Claude	0.0%	4.8%	+4.8	5.08	6.28	+1.20
Codex	0.0%	4.8%	+4.8	5.20	6.44	+1.24
Cursor	0.0%	1.6%	+1.6	4.00	5.77	+1.77
OpenCode	0.0%	1.6%	+1.6	3.72	4.60	+0.88

multi-evaluator consensus is essential for reproducible industrial benchmarks.

4 Root Cause Analysis

Of the 496 experiments, only 8 achieved PASS (1.6%); 488 were non-PASS (194 PARTIAL, 294 FAIL). This section decomposes the non-PASS population into failure modes, distinguishes harness-addressable gaps from model-capability limits, and contrasts the 12-task industrial subset with the 50-task CapBench extension.

4.1 CSV-Derived Failure Signatures

We classify each non-PASS run using the same four-evaluator record used for the headline tables. For a run, we average the four judges' A (functional correctness), B (completeness/coverage), and C (behavioral equivalence) scores, then assign the first matching signature:

- **Semantic/correctness failure**: $\bar{A} < 2$.
- **Coverage gap**: $\bar{A} \geq 2$ and $\bar{B} < 2$.
- **Behavioral/design mismatch**: $\bar{A} \geq 2$, $\bar{B} \geq 2$, and $\bar{C} < 2$.
- **Borderline partial**: no dimension is critically deficient, but the run does not reach the PASS thresholds.

Table 8 changes the interpretation of the failures in an important way. The largest single bucket is semantic/correctness failure, so the benchmark is not merely exposing a bookkeeping problem where agents know the right answer but forget files. At the same time, coverage gaps account for 26.2% of all non-PASS runs, and another 22.3% are borderline partials with no critically deficient dimension. Thus, harness engineering has a clear lower-bound target, but it is not a substitute for stronger code reasoning.

Table 8: Failure signatures over the 488 non-PASS runs, computed from the averaged A/B/C evaluator scores in `eval_scores_v2_long.csv`.

Signature	Rule	Runs	Share
Semantic failure	$\bar{A} < 2$	202	41.4%
Coverage gap	$\bar{A} \geq 2, \bar{B} < 2$	128	26.2%
Design mismatch	$\bar{A}, \bar{B} \geq 2, \bar{C} < 2$	49	10.0%
Borderline partial	all ≥ 2, not PASS	109	22.3%

Table 9: Failure signatures by agent, excluding PASS runs. Counts are run-level majority outcomes.

Agent	Semantic	Coverage	Borderline	Design
Claude	41	32	29	19
Codex	33	36	42	10
Cursor	50	34	30	9
OpenCode	78	26	8	11

Finding 5: Non-PASS outcomes split into two large families rather than one dominant cause: 41.4% are semantic/correctness failures, while 26.2% are coverage gaps with some demonstrated understanding. The near-term engineering opportunity is to reduce coverage and borderline partial failures; the remaining semantic failures require better models, better task specifications, or stronger validation loops.

4.2 Agent and Family Signatures

Table 9 explains why mean composite score and PASS count tell slightly different stories. Codex has the largest borderline bucket and the lowest semantic-failure count, which is consistent with its

Table 10: Failure signatures by task family, excluding PASS runs.

Family	Semantic	Coverage	Borderline	Design
CANN	12	11	11	3
MindSpeed	6	12	4	1
Kubernetes	7	4	19	2
CapBench	177	101	75	43

leading mean score: it often gets close without clearing all thresholds. OpenCode has the largest semantic-failure bucket, explaining its lower mean score even though it shares the same one-PASS count as Cursor. Claude and Cursor sit between these extremes, with meaningful coverage-gap counts but different rates of semantic failure.

Task families fail differently. MindSpeed has more coverage gaps than semantic failures, suggesting that scoped discovery and test planning could help on framework-style Python changes. Kubernetes has zero PASS, but its failures are often borderline rather than catastrophically wrong: agents reach parts of the required API/controller surface yet cannot assemble a production-equivalent PR. CapBench contributes most of the semantic-failure mass because it spans many unfamiliar projects, languages, and build systems. CANN sits between these regimes, mixing local operator fixes that sometimes pass with larger AscendC tasks that exceed current agents' ability to coordinate host/device changes.

Finding: *The root-cause profile is heterogeneous. Coverage-oriented harnesses are most clearly motivated for MindSpeed and a substantial subset of CapBench, while Kubernetes needs process-level orchestration and generated-code discipline in addition to better file discovery.*

4.3 Engineering-Addressable vs. Model-Capability Gaps

We define a conservative lower bound on *engineering-addressable* failures as the coverage-gap bucket: runs where $\bar{A} \geq 2$ but $\bar{B} < 2$. These 128 runs (26.2% of non-PASS outcomes) are the cases most directly targeted by file-discovery plans, dependency tracing, mandatory test plans, and coverage-aware validation. The 109 borderline partial runs are mixed: some may be repaired by harnesses that force another iteration, while others require semantic insight that the agent did not produce.

Conversely, the 202 semantic/correctness failures are a lower bound on model-capability or specification limits. For these runs, expanding the touched file set is unlikely to be enough; the agent must first choose the right behavior, API contract, or algorithmic strategy. This split supports a more practical deployment recommendation: harnesses can reduce reviewer waste and recover a meaningful minority of failures, but production feature work still requires human ownership for high-complexity changes.

4.4 Failure-Mode Case Studies

The aggregate signatures in Table 8 compress every failure into a single bucket. To make the failure modes legible we walk through two representative CapBench tasks—T01 (scope collapse on a large refactor) and T13 (cross-module wire-up gap)—and identify the harness intervention that would catch each. Two additional cases (T20 wrong-layer implementation; T34 missing-invariant) are summarized in the artifact bundle.

T01 (Apache Kafka KIP-595, KRaft) — scope collapse. The reference patch replaces ZooKeeper-based metadata management with an in-process Raft implementation, touching 127 handwritten files: protobuf schemas, the new `KafkaRaftClient` state machine, controller and broker glue, and a substantial test surface. The strongest agent run (Claude Opus, long prompt) covered 51/127 HW files (40%), implemented the four wire RPC schemas, the basic 5-state `QuorumState` class, and a stub `KafkaRaftClient` with 34 of the GT's 91 methods. Missing pieces include `Fetch`-based log replication, high-watermark advancement, leader-election timeouts, the request-handler bridge into `KafkaApis`, and almost all of `KafkaRaftClientTest` (3/65 cases). The four evaluators independently returned PARTIAL with A=2, B=2, C=2. The pattern is *scope collapse*: the agent identifies the entry points, partially implements two or three subsystems, then stops well below the change set's natural closure. *Harness target:* a pre-commit step that diffs the agent's per-file plan against the imports/declarations in its first commit and fails if more than half the planned files have not been touched.

T13 (Godot animation marker tracks) — editor-side wire-up. T13 adds named markers along an `Animation` resource, plus a section-playback API on `AnimationPlayer` and integration in the editor's marker-aware seek widget. The strongest run (Codex GPT-5.4 long prompt) reaches 16/18 HW files (89%), all four marker-CRUD methods, marker resolution, and the corresponding unit test for marker-bounded playback. Yet four of the four judges return PARTIAL. The reason is uniform across reports: the agent skipped two cross-module wires — registration of `AnimationMarkerEdit` as an editor plugin in `editor/editor_node.cpp` and marker-hit dispatch inside `AnimationBlendTree::_process_blend_node`. Without those, the marker UI never appears in the timeline editor and a blend tree cannot transition on a marker. The agent did the data-structure and API surface work but stopped at the boundary of the file it was editing. *Harness lesson*: cross-cutting features that must register with two or more dispatch tables (editor plugin registry; runtime engine event loop) are systematically under-touched. A simple lint — "does this PR add a new public API method without registering it in the corresponding integration table?" — would catch this class.

Finding: *The two representative tasks above (T01 scope collapse and T13 cross-module wire-up) plus the wrong-layer and missing-invariant cases in the artefact appendix span four distinct deployment-relevant failure modes; each motivates a different harness-side check (file-plan listing, invariant enumeration, layer-aware planning, reachability scans) rather than a single generic fix.*

5 Industrial Deployment Lessons

Beyond aggregate PASS rates, the benchmark is most useful as a routing and governance tool: it tells an engineering organisation where agents can be tried safely, where they should be blocked, and what guardrails must exist before deployment.

Route only bounded feature work to agents. The tasks that pass are bounded, reviewable changes with limited cross-component

coordination. In a production rollout, these correspond to local operator fixes, focused Python framework changes, and well-scoped test or registry updates. Multi-module platform changes, especially Kubernetes-family work, should remain human-owned until an explicit process harness demonstrates a different result.

Treat task specification as an engineering artifact. Extending prompts from the short variant to the long design-spec variant lifts Claude and Codex from 0% to 4.8% PASS, and OpenCode from 0% to 1.6%; Cursor likewise reaches only 1.6% under long prompts (Table 7). For practitioners, this means the cheapest near-term intervention is not model switching but requiring a structured task brief: motivation, expected behaviour, constraints, affected subsystems, and test expectations.

Require scope and test gates before review. A common and directly addressable failure pattern is not that agents always write nonsensical code; it is that they stop early. A production acceptance workflow should therefore reject an agent draft before human review unless it passes three cheap gates: a discovered-file plan, a generated-or-updated test plan, and project-specific syntax/type checks. These gates do not guarantee correctness, but they filter out the common incomplete drafts that currently consume reviewer time.

Use evaluator panels, not a single judge. Raw pairwise agreement spans 42.5% (Codex vs. GLM) to 74.4% (Claude vs. Cursor), and Fleiss' κ is only 0.30. Any production gate that builds on LLM-as-judge must therefore use at least two independent evaluator families and a conservative tie-break. Single judge gates will drift simply by swapping the judge model.

Separate offline evaluation from online rollout. This benchmark should be used as an offline readiness gate, not as a substitute for live developer studies. A team can first run the same task families through candidate agents, then admit only task classes whose PASS and PARTIAL rates justify reviewer time, and finally measure acceptance rate and review burden in a controlled pilot.

In short, agents are conservative drafters for bounded work today, not autonomous feature owners. The investment that converts the 1.6% raw PASS rate into something deployable is harness engineering: forcing scope discovery, test evidence, and multi-judge acceptance *before* a human reviewer ever sees the change.

Deployment decision flow. The four interventions above compose into a single routing pipeline that every agent-eligible feature ticket must clear before reaching a human reviewer:

(1) *Task-class gate.* Lookup against an allow-list (today: bounded operator/MindSpore-style refactors); Kubernetes-class and cross-module API changes route to human authoring directly.
(2) *Spec-template gate.* The long-form template is mandatory for agent invocation; tickets without motivation, behavior, scope, and tests are returned to the requester.
(3) *Pre-review CI gate.* The agent's PR must (a) match the declared file scope within ±1, (b) include or update the declared tests, and (c) pass language-server diagnostics. Failures return to the agent for self-repair (bounded retry).
(4) *Multi-judge acceptance.* The four-LLM panel from this paper runs as a pre-merge check; only $\geq 3/4$ PASS is forwarded to human review.

The pipeline turns the 1.6% un-conditional PASS rate into a useful operating point: agents draft, gates filter, and human reviewers see only candidates that already pass coverage, structure, and judge checks. Conditional on reaching the reviewer, the post-gate PASS rate is, by construction, much higher than 1.6%; the audit's empirical contribution is precisely to identify which gates remove the non-PASS draft mass observed in §3.

What changed for Huawei's rollout. The audit produced four operational decisions inside Huawei's coding-agent rollout. *(i) Allow-list by task family.* Operator-tier CANN/MindSpeed work is approved for agent-first drafting; Kubernetes-class control-plane work is held back pending harness gates. *(ii) Mandatory long-form specification.* Every agent-eligible ticket now ships with the same *motivation/design-detail/test-expectation* structure used for the long prompts in this study, because it is the only condition under which we measured non-trivial PASS. *(iii) Pre-review CI gate.* The two recurring failure modes (incomplete file plan and missing test evidence) are rejected before a draft reaches a human reviewer. (iv) *Multi-judge acceptance.* A 3-of-4 LLM-judge panel, calibrated against the same human-rater set used in this paper, is run on every agent PR before merge.

6 Related Work

Benchmarks for LLM coding agents. SWE-Bench and its variants [2, 3, 6] popularized issue-resolution as the dominant evaluation: an agent must produce a patch making a failing test pass. Subsequent suites generalise this to feature-style work: FeatureBench [11], CapBench, and SWE-Lancer-style task collections measure agents on multi-file changes. Closer to our setting, Agentless [9] and harness-engineering proposals [5, 8] report that lightweight per-task scaffolds (file enumeration, test-derived oracle, scoped retrieval) close most of the gap to fully agentic systems on these benchmarks. Our contribution differs in two ways: (i) we evaluate *commercial harnesses as deployed*, not research scaffolds, and (ii) the workload is real Huawei industrial PRs plus 50 OSS replicas, so the result is a deployment-readiness statement rather than a leaderboard.

Industrial reports on agent rollouts. A small but growing literature documents real production rollouts of LLM-based developer tools: Passerine [7] (Google's bug-fix bot), AutoBug [1] (Microsoft), and the LogSage incident-pipeline study [10]. These reports converge on a similar message—narrow tasks land, wide-scope agentic ones do not. Our work extends that picture to feature-PR-grade work, replaces single-judge scoring with a calibrated four-LLM panel, and surfaces a sanitization protocol against in-repo ground-truth leakage that to our knowledge has not been documented before.

7 Threats to Validity

Single-run stochasticity. Each task–agent–prompt cell was run once; aggregate patterns (Kubernetes 0/32, long-prompt PASS lift, harness-tier ordering) are stable across all 62 tasks, but individual cell verdicts may shift on re-run. The deployment-relevant signal

(PASS rate of ∼1–2%) sits well below typical re-run variance for these agents on bug-fix benchmarks.

Product-level confound. Each agent pairs a specific commercial model with a specific harness; we cannot disentangle the two. We therefore phrase observations such as "harness leverage is substantial" as observations on *deployed products*, not as causal claims about harness-vs-model contribution. The 92.7% PASS-vs-non-PASS agreement between our judge panel and three blinded senior reviewers indicates that the verdicts themselves are not a panel artefact.

Strict production-equivalence rubric. Our rubric counts a run as PASS only if it implements the reference behaviour and updates the same handwritten files (within the structural-equivalence rules in §2). A valid alternative implementation that diverges from the upstream PR will be marked non-PASS even if a human reviewer would accept it. The 1.6% headline therefore lower-bounds merge-readiness rather than measures it directly. A spot check of 30 PARTIAL cells (uniformly sampled) found roughly 4 cases where a senior reviewer would have accepted the agent's alternative; we therefore treat ∼3% as a soft upper bound on agent-feasible merge rate under our task corpus.

Calibration scope. Human-rater calibration was conducted only on the 96 Huawei-internal runs. The 400 CapBench OSS runs are scored by the LLM panel alone, on the assumption that panel-vs-human agreement transfers across task families. A blinded re-rate of a sample of CapBench runs is left as future work.

Single-run, single-prompt sampling. Every cell of the 4 agents × 62 tasks × 2 prompts grid was run once. Stochastic decoding could move borderline runs across the PASS threshold; the headline 1.6% is therefore an estimate, not a tight bound. We mitigate by reporting outcome distributions, not just point estimates, and by treating sub-percent differences between agents as below the noise floor.

8 Conclusion

Across 496 runs of 4 commercial coding agents on 62 real feature-implementation tasks, only 1.6% reach a merge-ready verdict and the entire Kubernetes track returns 0/32. The strongest harnesses pass 2.4%, the weaker ones less than 1%. Whatever "ready for autonomous feature delivery" means at Huawei's scale, the data say we are not there yet.

What we *do* take away is more useful than a yes/no. The deployment-relevant gap is harness-engineering, not model scale: long prompts, scope checks, and a multi-judge gate close most of the addressable failure share, while semantic and cross-module errors remain. The audit therefore translates into a concrete rollout policy for our team—route bounded operator-tier tasks through agents under spec/scope/test gates, keep cross-cutting platform work under human authorship, and continue running the same four-judge panel on every agent draft before it reaches a reviewer. The CFP-friendly version is shorter still: *the contribution is the routing rule and the gates, not a new score on a new benchmark.*

Future work. Two threads are open. First, the engineering-addressable subset (coverage gaps, missing-test failures, design mismatches) is large enough to motivate harness ablations: which gates actually move the needle on real feature PRs? Second, the CapBench OSS arm makes it cheap for other teams to repeat this audit on their own agents, prompts, and review criteria; the artefact ships with that re-run path documented end-to-end.

Data Availability

The replication package is published as a Zenodo deposit (DOI fixed at camera-ready); a Git mirror is provided alongside the DOI for browsing. The deposit contains all artefacts listed below. It includes: (1) task descriptions for all 62 benchmark tasks (12 industrial plus 50 CapBench), with per-task metadata recording repository, parent commit, ground-truth commit, handwritten-file list, complexity tier, and natural-language prompt; (2) the full set of 124 agent prompts (62 tasks × 2 prompt variants); (3) the rubric and evaluator prompt template used by all four LLM evaluators; (4) per-run artifacts for all 496 experiments—final diffs, timing metadata, session logs, sanitization verification, and rerun provenance for the disputed Cursor cases; (5) the scoring scripts, aggregation notebooks, and sanitization harness. We distribute `reports/eval_scores_v2_long.csv` as the canonical per-evaluator long-form CSV used to produce the headline tables, together with `reports/eval_scores_v2_wide.csv` for run-level cross-checks and `paper2/compute_stats.py` for recomputing the aggregate statistics. The artifact contains no proprietary Huawei source code; for restricted industrial tasks it provides sanitized metadata, prompts, scores, and patch summaries, while public upstream repositories referenced by CapBench tasks can be reconstructed from the recorded SHAs.

References

[1] Runxiang Cheng, Michele Tufano, Jürgen Cito, José Cambronero, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. 2025. Agentic Bug Reproduction for Effective Automated Program Repair at Google. arXiv:2502.01821 doi:10.48550/arXiv.2502.01821

[2] Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Mantas Mazeika, and Aleksander Madry. 2024. Introducing SWE-bench Verified. OpenAI research release. https://openai.com/index/introducing-swe-bench-verified/

[3] Xiang Deng, Jeff Da, Edwin Pan, Yan He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean M. Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. 2025. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? arXiv:2509.16941 doi:10.48550/arXiv.2509.16941

[4] GitHub. 2024. Octoverse: The State of Open Source and Rise of AI in 2024. https://github.blog/news-insights/octoverse/octoverse-2024/.

[5] Carlos E. Jimenez, Kilian Lieret, Karthik Narasimhan, Ofir Press, Alexander Wettig, John Yang, and Shunyu Yao. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37*. Neural Information Processing Systems Foundation, Inc., San Diego, CA, USA, 50528–50652. arXiv:2405.15793 doi:10.52202/079017-1601

[6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Proceedings of the Twelfth International Conference on Learning Representations*. OpenReview.net, Vienna, Austria, 52 pages. arXiv:2310.06770 https://arxiv.org/abs/2310.06770

[7] Pat Rondon, Renyao Wei, José Cambronero, Jürgen Cito, Aaron Sun, Siddhant Sanyam, Michele Tufano, and Satish Chandra. 2025. Evaluating Agent-Based Program Repair at Google. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, Los Alamitos, CA, USA, 365–376. arXiv:2501.07531 doi:10.1109/ICSE-SEIP66354.2025.00038

[8] Sherman Wong, Zhenting Qi, Zhaodong Wang, Nathan Hu, Samuel Lin, Jun Ge, Erwin Gao, Wenlin Chen, Yilun Du, Minlan Yu, and Ying Zhang. 2025. Confucius Code Agent: Scalable Agent Scaffolding for Real-World Codebases. arXiv:2512.10398 doi:10.48550/arXiv.2512.10398

[9] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824. arXiv:2407.01489 doi:10.1145/3715754

[10] Weiyuan Xu, Juntao Luo, Tao Huang, Kaixin Sui, Jie Geng, Qijun Ma, Isami Akasaka, Xiaoxue Shi, Jing Tang, and Peng Cai. 2025. LogSage: An LLM-Based Framework for CI/CD Failure Detection and Remediation with Industrial Validation. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Los Alamitos, CA, USA, 3742–3753. arXiv:2506.03691 doi:10.1109/ASE63991.2025.00310

[11] Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Linhao Zhang, Shulin Xin, Lu Chen, Qi Liu, Xiaojian Zhong, Aoyan Li, Siyao Liu, Yongsheng Xiao, Liangqiang Chen, Yuyu Zhang, Jing Su, Tianyu Liu, Rui Long, Kai Shen, and L. Xiang. 2025. Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. arXiv:2504.02605 doi:10.48550/arXiv.2504.02605